

CONTAINERS



Work Integrated Learning Programmes

MOHANA MURALI GURUNATHAN

AGENDA

Containers

VMs and Containers, Analogies

Docker



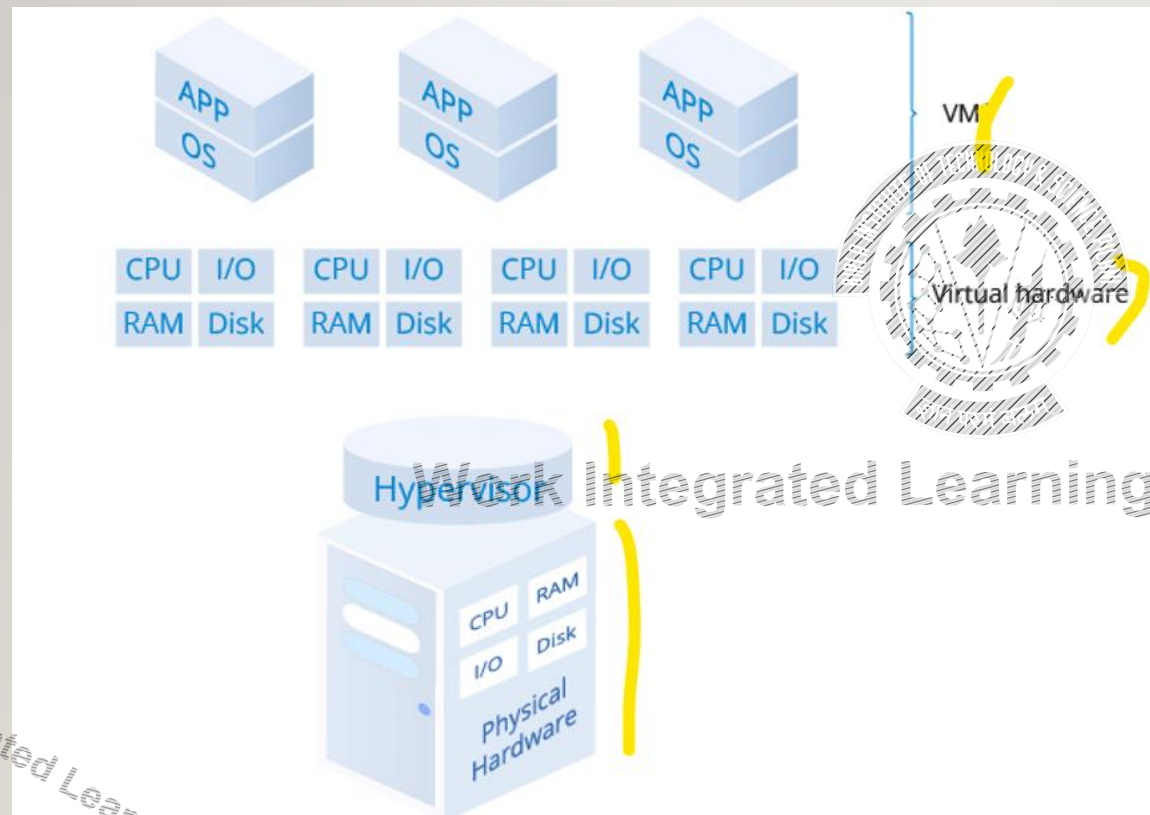
Work Integrated Learning Programmes

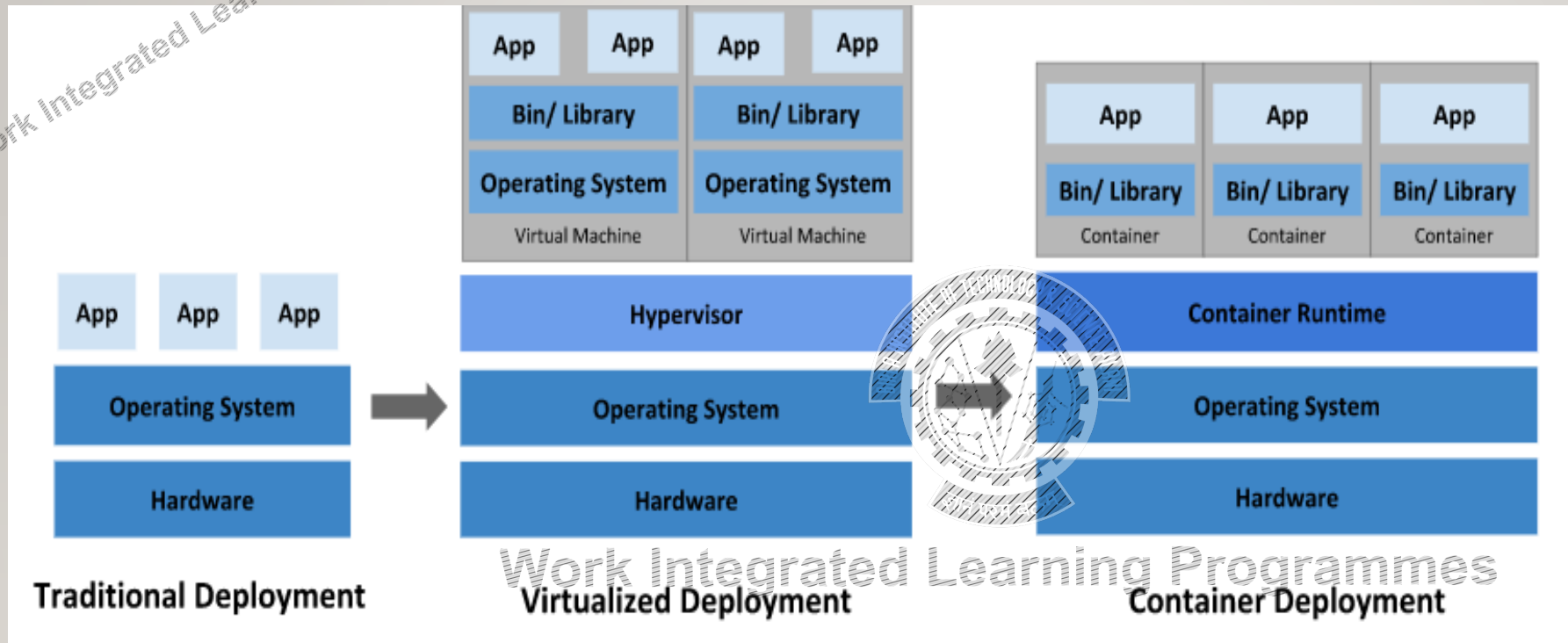
CONTAINERS

- Lightweight virtualization mechanism
- A software container is a standardized package of software.
- Everything needed for the software to run is inside the container
- The software code, runtime, system tools, system libraries, and settings are all inside a single container
- Managed by the OS kernel running on the host system
- Isolated memory, CPU, storage, process table, and networking interfaces
- Faster provisioning for newer applications

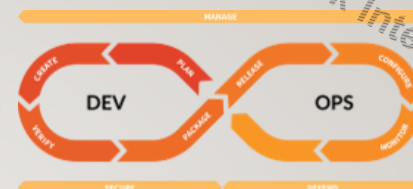


BACK TO BASICS! – VIRTUALIZATION





Work Integrated Learning Programmes



Any App

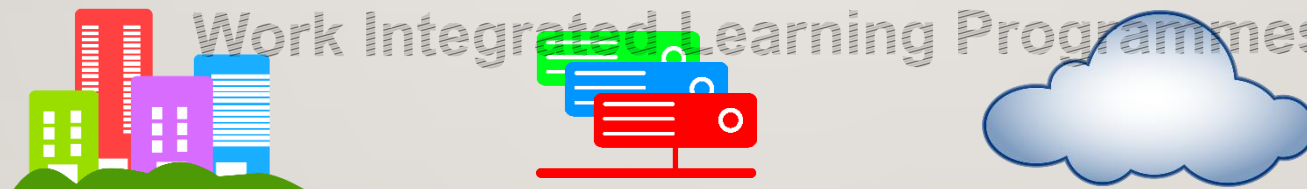


CONTAINERIZATION ENGINE

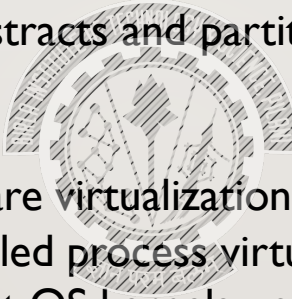
Any OS



Anywhere



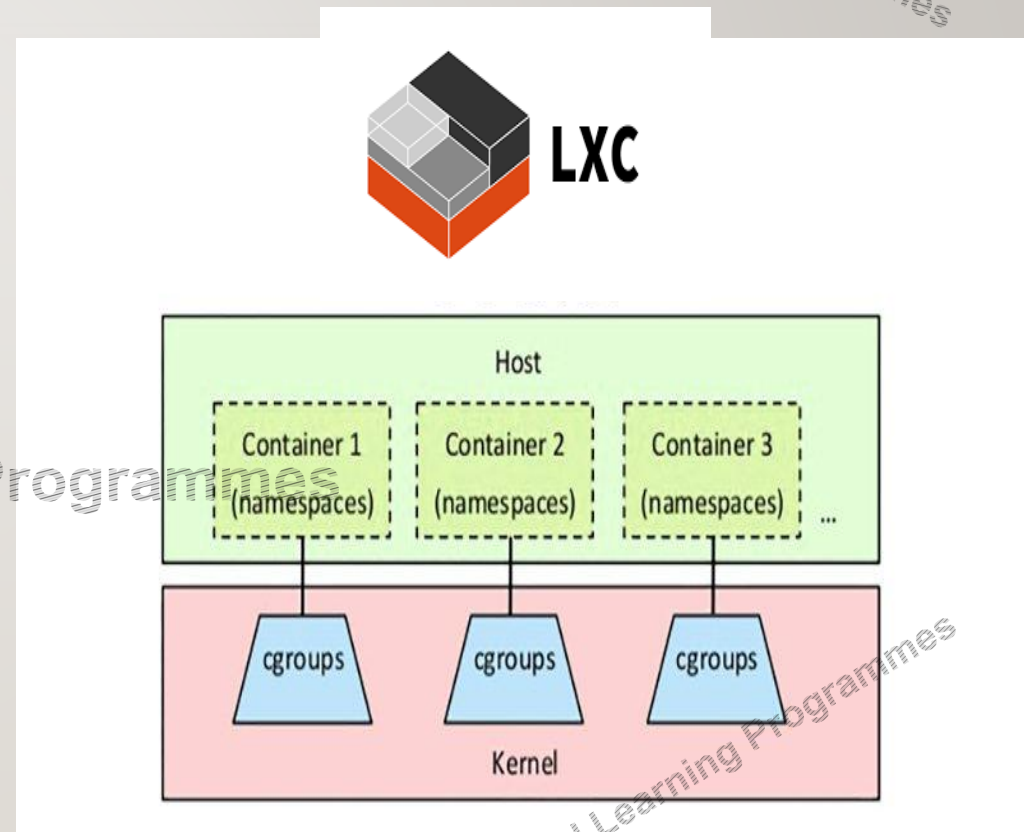
Virtual Machines (VMs): A VM is a software-defined environment managed by a hypervisor that partitions physical hardware into isolated systems, leveraging hardware-assisted virtualization for near-native performance. VMs are classified under **hardware virtualization** because the hypervisor abstracts and partitions the **physical hardware** (CPU, memory, storage, network).



Containers: Containers are not classified under hardware virtualization as they do not emulate hardware or run separate kernels. Instead, they use OS-level virtualization (sometimes called process virtualization or namespace isolation). The container runtime (e.g., Docker, Containerd) leverages features of the host OS kernels - such as Linux namespaces and cgroups, or Windows OS isolation mechanisms - to isolate processes and resources. All containers share the host kernel while running independently with their own filesystems, libraries, and dependencies.

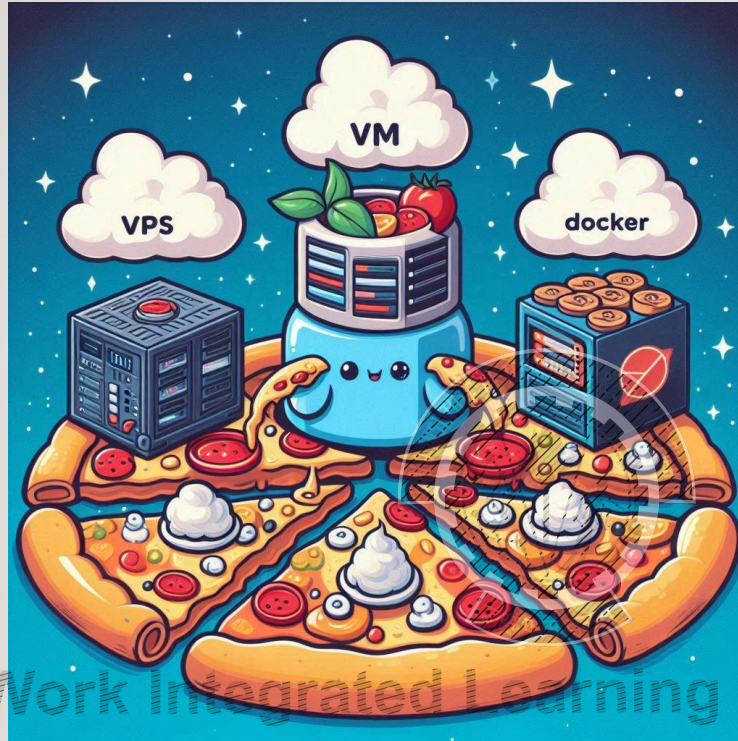
TWO MOST IMPORTANT ASPECTS FOR CONTAINERS

Technology	Primary Function	What it Provides
Namespaces	Isolation	A container's own separate view of system resources (e.g., PID 1, network stack, filesystem mount points).
Cgroups (control groups)	Resource Management	Limits and allocates system resources (CPU, memory, disk I/O, etc.) to prevent any single container from monopolizing the host system. A kernel mechanism for limiting and measuring the total resources used by a group of processes running on a system



<https://www.youtube.com/watch?v=xInpPrzyKfs>

an in-depth look at the foundational Linux technologies that make containers possible, including namespaces and cgroups:



Let's use an Analogy – Pizza Shop

The pizza shop: The physical server



Assume that this shop has all the hardware needed to make pizzas: ovens, counters, preparation stations & ingredients. This whole shop represents a single, powerful physical server.

Let's use a Pizza Shop as Analogy

VM – A PRIVATE BOOTH WITH ITS OWN KITCHEN



- The booth comes with its **own fully-equipped, separate mini-kitchen** (its own guest Operating System). This includes a mini-oven, separate utensils and a preparation area.
- You are completely **isolated** from other customers. What happens in their booth (e.g. someone starts a small fire in their oven) does not affect your booth at all.
- The trade-off is that setting up and running each booth with a full kitchen is **resource-intensive** and takes more effort.

The Hypervisor: The Booth Manager who allocates electricity, gas, and space to each mini-kitchen.

CONTAINERS – A STANDARD DELIVERY BOX



- A Container is a much more lightweight and portable solution. It's like a perfectly packaged pizza delivery box.
- The box contains everything we need to enjoy the food: the pizza, plates, paper towels, plastic knives and small sachets of oregano, chilli flakes etc.
- We don't need a whole new kitchen (guest OS) to enjoy it. All we need is a table to set the box on, which can be any of the prep stations or booths.
- The delivery service (the container runtime, like Docker) ensures the box is sealed and identical no matter where it's opened.
- Multiple boxes can be opened on the same counter (server), making it much more **efficient** than giving every customer a full kitchen.

Container Engine The Delivery Counter Staff who efficiently receives, arranges, and sets out the standardized boxes on the shared counter.

Concept	Pizza Shop Analogy	Technical equivalent/parallel
Physical Server	The entire pizza shop with all its resources.	A single powerful computer
VM	A private booth with its own kitchen	A Guest on a hypervisor
Container	A standardized pizza delivery box	An isolated application environment sharing the host OS

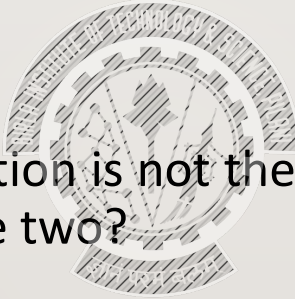


Work Integrated Learning Programmes

SUMMARY & KEY DIFFERENCES IF WE USE A DIFFERENT ANALOGY – SAY **APARTMENTS**

Concept	Standard Apartment Analogy
Physical Server	The entire apartment building
Virtual Machine	A fully independent townhouse: Has its own private plumbing, electrical grid, and heating system (Guest OS kernel).
Container	A private room in a co-living apartment: Has its own lock and key (Process isolation), but shares the central plumbing, heating, and electric main (Host OS kernel).

QUESTIONS – LET'S DEBATE!



If the concept of Virtualization is not there, is it possible to have the existence of these two?

Can both of these two co-exist in a single physical machine?

IF VIRTUALIZATION WAS NOT THERE...



It's Impossible to have the existence of these two without virtualization
– as both provide(create the illusion of) isolated environments.

VMs – Virtualize hardware & Containers – Virtualize the OS

CAN THEY CO-EXIST IN A SINGLE PHYSICAL MACHINE?

Imagine that we have a powerful physical server with the following setup and Configs

- Host OS: Ubuntu Server
- Hypervisor: KVM (Kernel-based Virtual Machine)



Configuration:

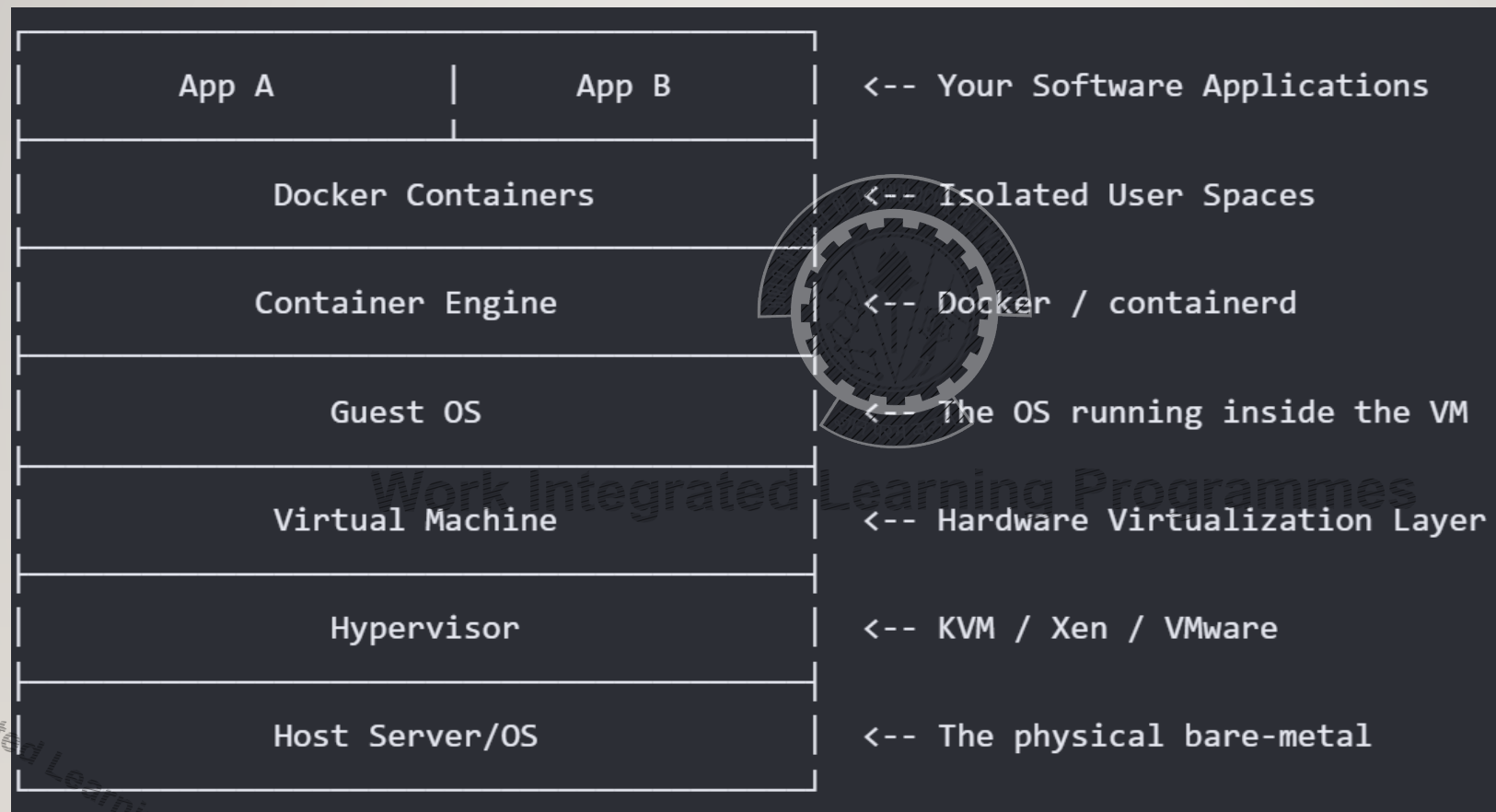
VMs: On the same host, we can create additional VMs for different purposes. For example, one VM could run a Windows Server for a specific application, while another VM runs a different Linux distribution for a legacy system.

Docker: Docker is installed on the host OS. We can run multiple Docker containers for microservices, such as a payment gateway, user authentication service and inventory management system etc. A typical local development environment.

All in one – VMs & Dockers co-existing parallelly !!!

CAN CONTAINERS RUN INSIDE A VM?

YES



Running a container on AWS EC2, Google Cloud, or Azure.

Container engine (like Docker) is running inside a Virtual Machine.

Containers not only can run inside a VM, but doing so is the standard deployment model for almost all major cloud providers

Work Integrated Learning Programmes

Work Integrated Learning Programmes

CAN A VM RUN INSIDE A CONTAINER? YES



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Activity: Research

CO-EXISTENCE BENEFITS

- Resource Utilization: Efficiently use the physical server's resources by running different workloads in isolated environments.



- Flexibility: Easily manage and scale applications using containers, while maintaining isolated environments for specific needs with VMs.

Work Integrated Learning Programmes

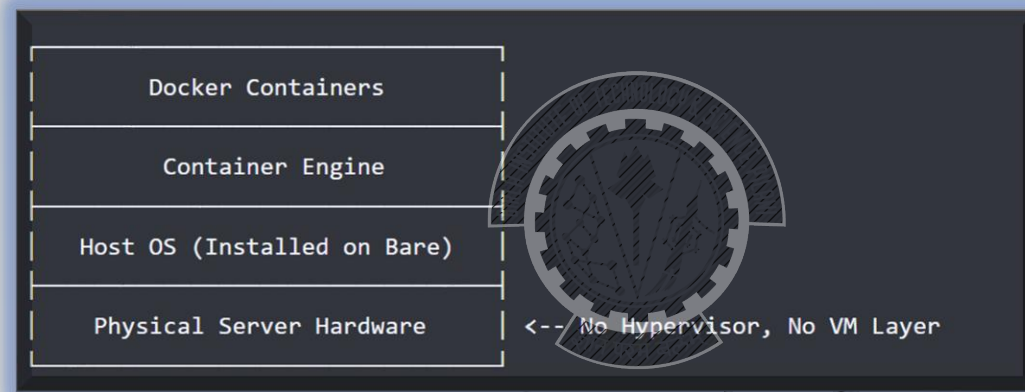
- Cost-Effective: Reduce hardware costs by consolidating multiple services on a single physical machine.
- Such setups allow us to leverage the strengths of each technology, ensuring efficient resource utilization, scalability, and security.

All in one!!!

CONTAINERS RUNNING WITHOUT A VM IN THE CLOUD

All major cloud providers give us the option to rent an entire physical, raw server with no hypervisor and no virtual machine layer underneath it. Example: **AWS**: EC2 Bare Metal instances (e.g., m5.metal, c5.metal)

When we run Docker on these specific instances, the containers are running directly on the physical hardware.



Bare Metal instances:

- The cost would be a premium.
- Elimination of “noisy neighbours”.
- Advanced performance (as there is no hypervisor)

DOES AWS ECS & FARGATE USE BAREMETAL OR VM

They do not use Bare Metal instances but rather VMs for a couple of reasons

Research

Fargate also has the concept of MicroVMs – study activity.



Work Integrated Learning Programmes

Activity: Research

DOCKER



- An open platform for developing, shipping, and running applications.
- Separate your applications from the infrastructure (from each other also within the same VM)
- Enabler for quick delivery of Software.
- By taking advantage of Docker's methodologies for shipping, testing, and deploying code, we can significantly reduce the delay between writing code and running it in production.

Isn't this awesome?!

PROBLEM STATEMENT

- Let's assume that we are developing a killer web app that has three main components - a Node.js frontend, a Python API and a PostgreSQL database. If you wanted to work on this project, you'd have to install Node, Python, and PostgreSQL.
- How do we make sure of the following:
 - you have the same versions as the other developers on your team? Or your CI/CD system? Or what's used in production?
- How do you ensure the version of Python (or Node or the database) your app needs isn't affected by what's already on your machine? How do you manage potential conflicts?

Conflicts!!!

SOLUTION.... CONTAINERS

- Containers are isolated processes for each of your app's components. Each component - the frontend app, the Python API engine, and the database - runs in its own isolated environment (100% isolation!)

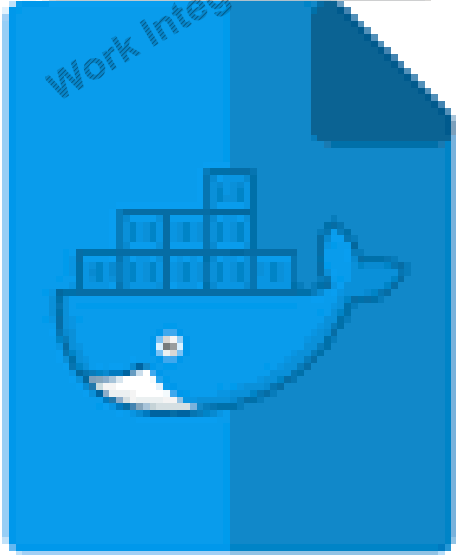
Here's what makes it awesome. Containers are

- **Self-contained** Each container has everything it needs to function with no reliance on any pre-installed dependencies on the host machine.
- **Isolated** Since containers are run in isolation, they have minimal influence on the host and other containers, increasing the security of your applications.
- **Independent** Each container is independently managed. Deleting one container won't affect any others.
- **Portable** Containers can run anywhere! The container that runs on your development machine will work the same way in a data center or anywhere in the cloud!

Again .. Isn't this awesome?!!!!

BIPO-WILP | Generated: 30-Aug-2020 15:06:15

Dockerfile



Build



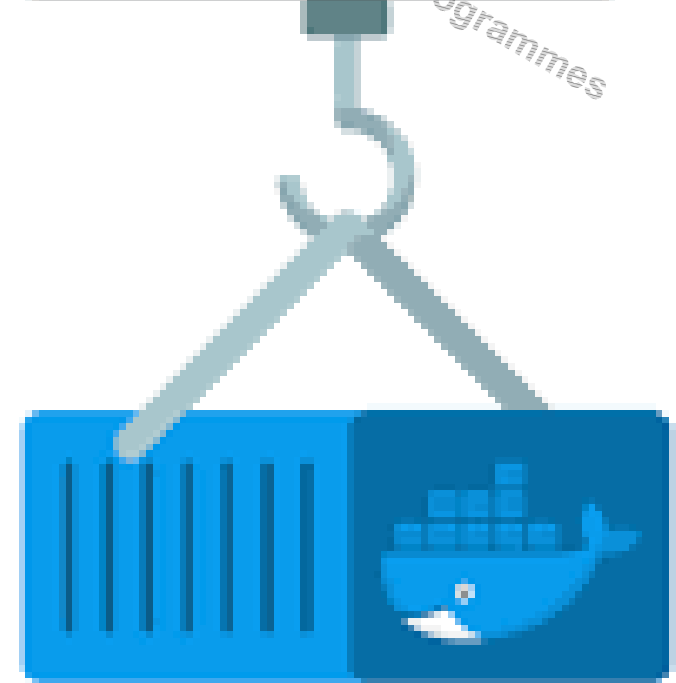
Docker Image



Run

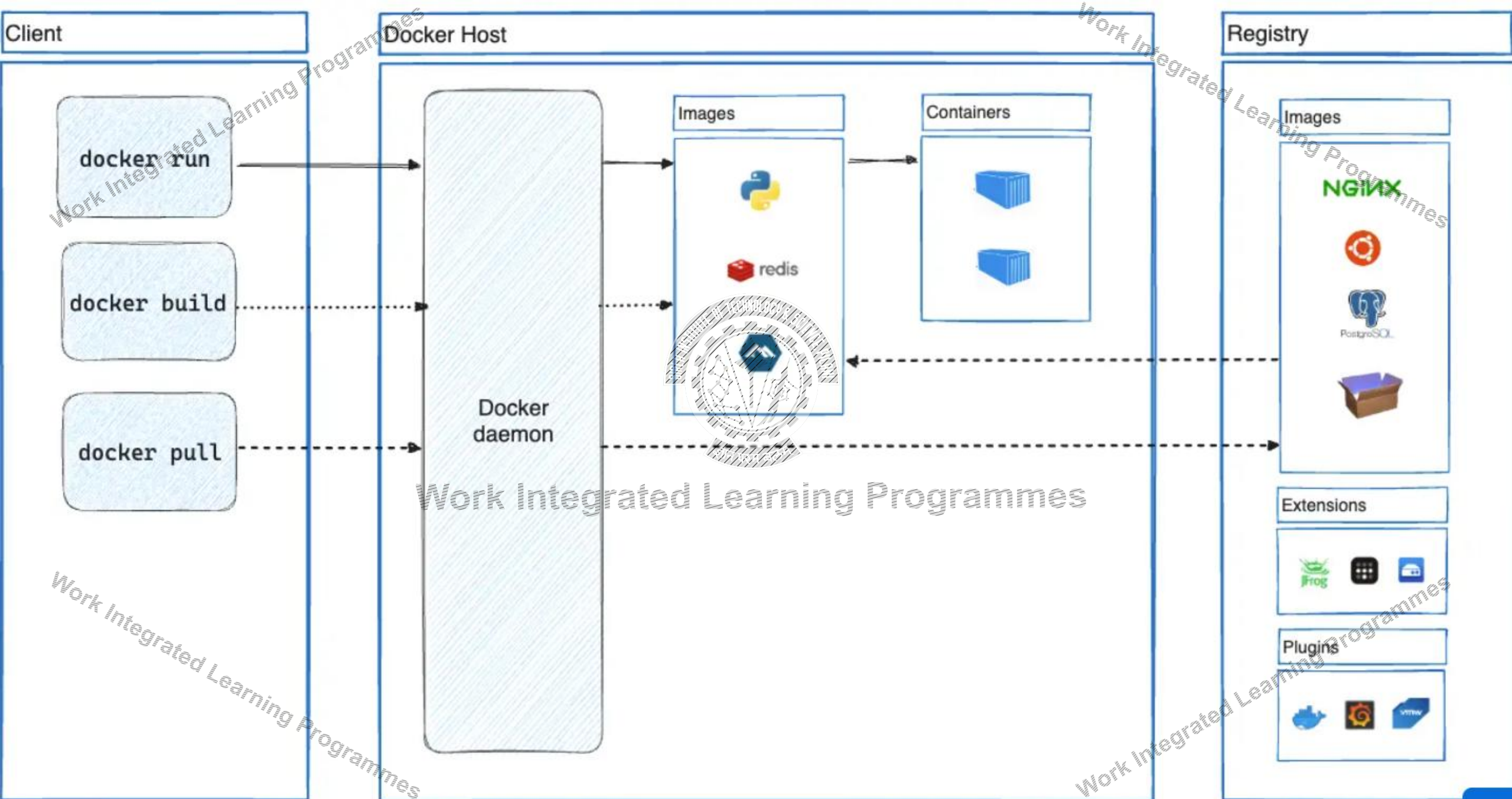


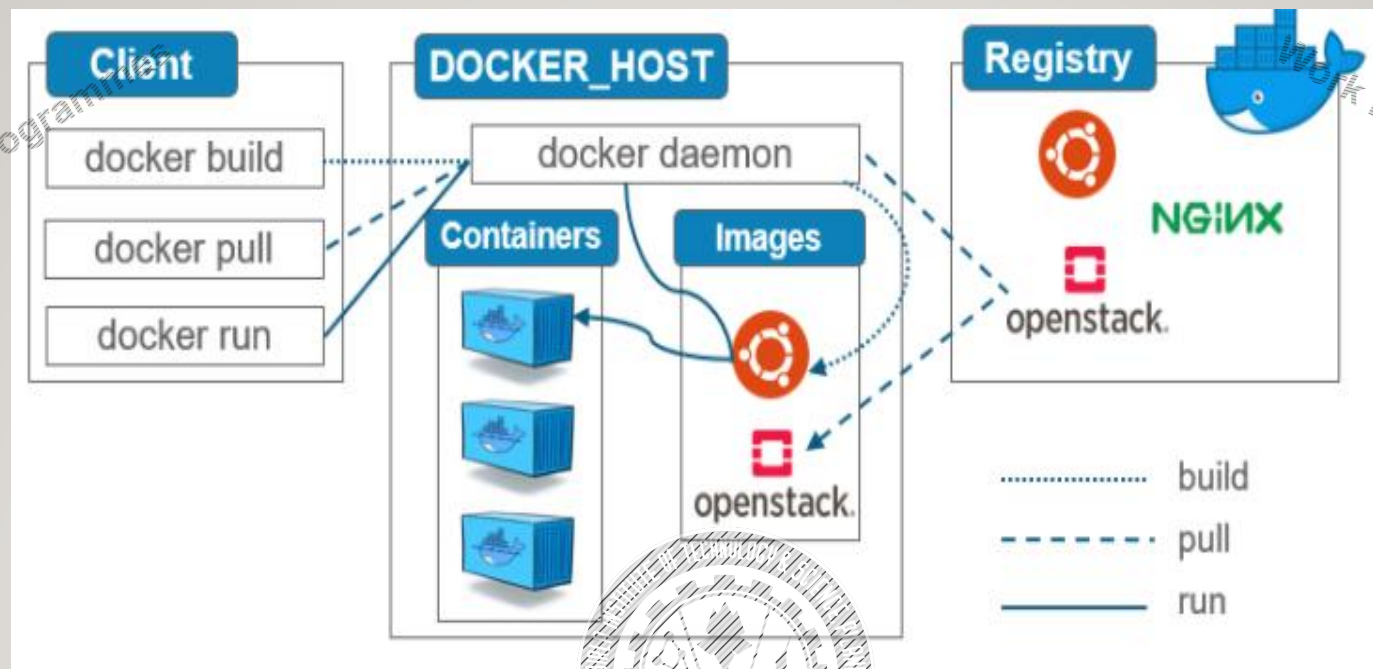
Docker Container



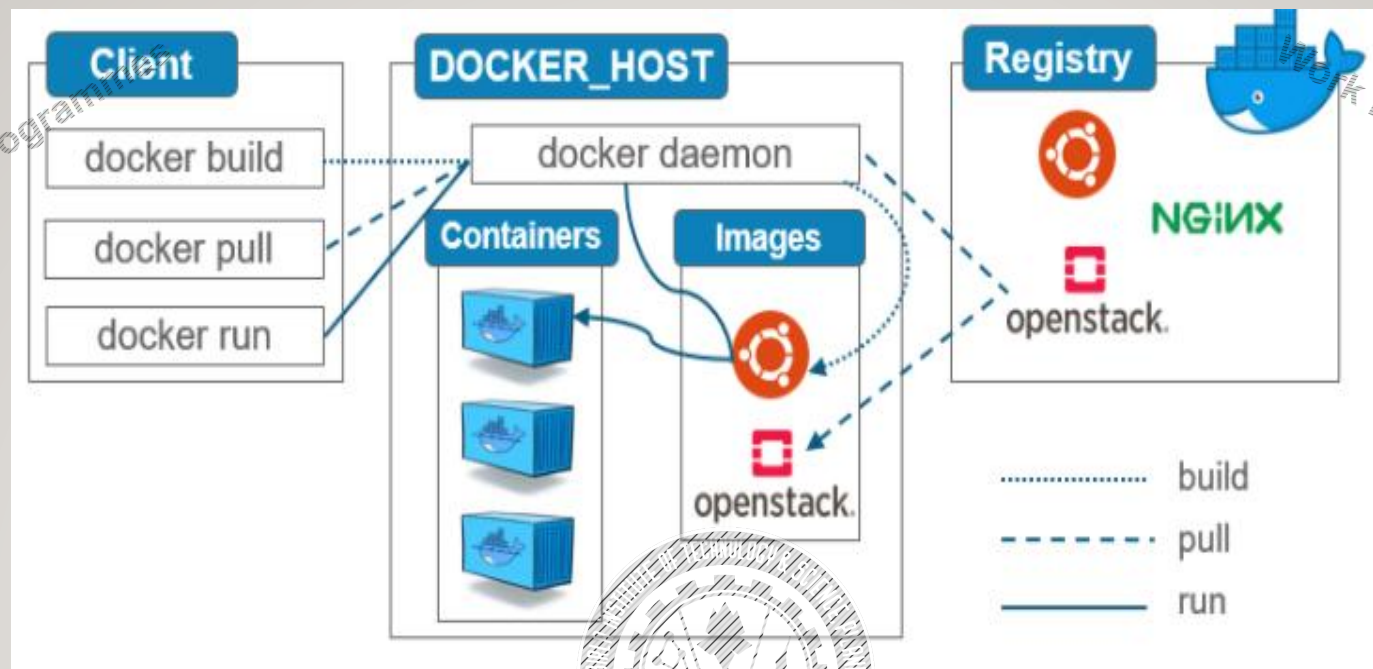
Typical Sequence

BPIS WILP | Generated: 30-Aug-2020 15:05:15



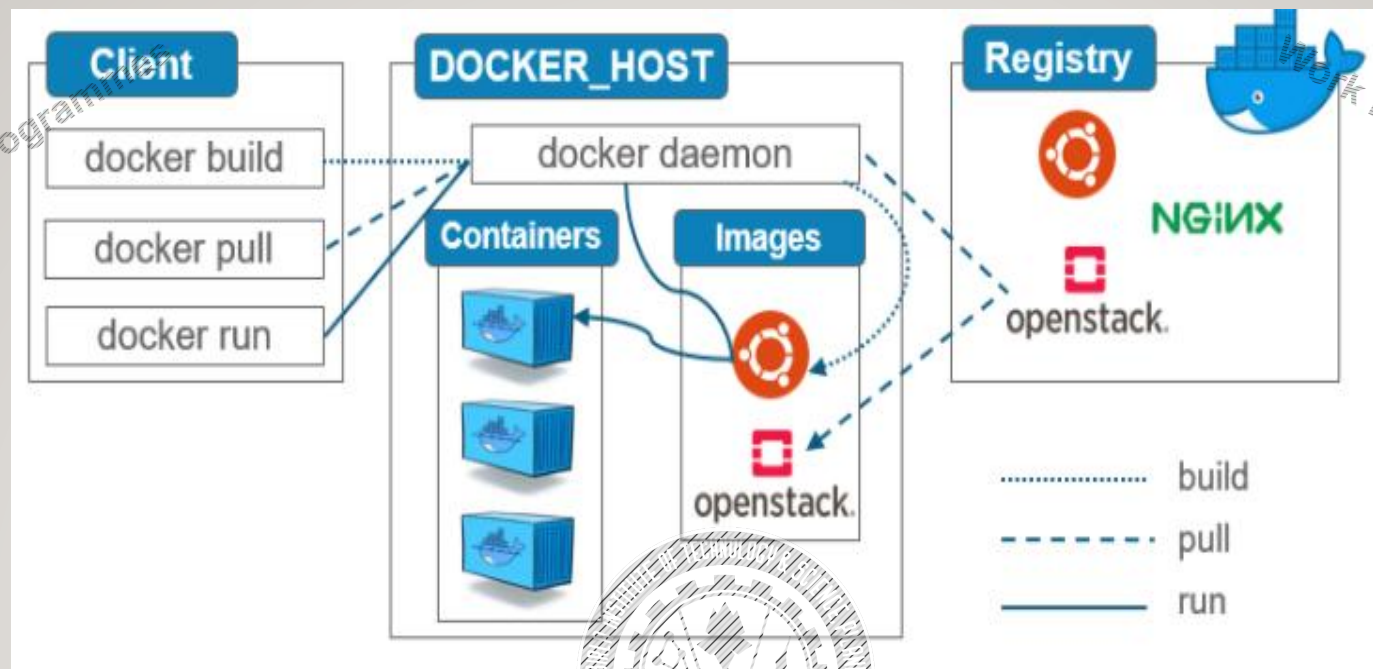


- Docker provides tooling and a platform to manage the lifecycle of the containers
 - Develop application(s)
 - Distribute & test
 - Deploy into production environment, as a container or an orchestrated service.
- Containers are great for continuous integration and continuous delivery (CI/CD) workflows.



Client-server architecture

- The Docker daemon
- The Docker client
- The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.



Docker Registries

Stores Docker images.

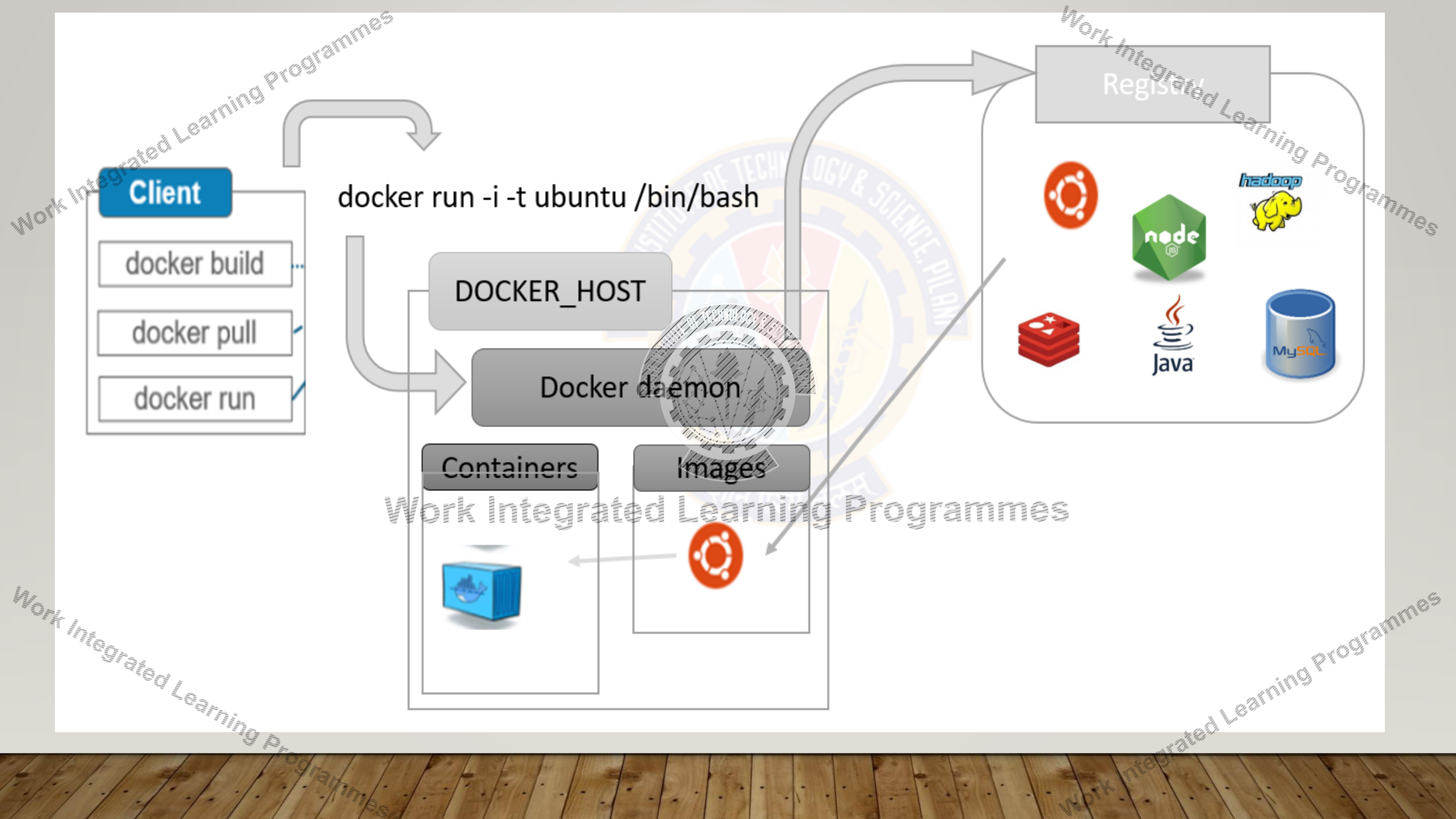
Docker Hub is a public registry that anyone can use.

Docker is configured to look for images on Docker Hub by default

Docker Objects

IMAGES: An image is a read-only template with instructions for creating a Docker container. Images are **immutable**.

CONTAINERS: A container is a runnable instance of an image. Containers are **mutable**



A container is a runtime instance of a docker image

Create and run a container from an image, with a custom name:

```
docker run --name <container name> <image name>  
docker run --name mylinuxserver Ubuntu
```

Run a container with the port number that can be used to access the same from outside the docker network.

```
docker run -p <host port>:<container port> <image name>  
docker run -p 8080:80 nginx
```

Run a container in the background

```
docker run -d <image name>  
docker run -d nginx
```

Note: Always provide a name (--name). Otherwise docker would assign a random name. If a port mapping is not provided, then the container cannot be accessed from outside the docker network.

Start or stop an existing container:

`docker start/stop <container name> (or <container id>)`

`docker stop id`

Remove a stopped container:

`docker rm <container name> (or <container id>)`

`docker rm -f mynginx`

`-f` : forcefully removes one or more running containers from your system.

Open an interactive shell inside a running container (for debugging purposes)

`docker exec -it <container name> sh`

`docker exec -it myubuntu bash`

Get the logs of a container: **`docker logs -f <container name>`** (streaming live logs)

To inspect a running container: **`docker inspect <container id> (or ) <container name>`** (queries the low level configuration metadata of a Docker object from the host).

To list currently running containers: **`docker ps`**

List all docker containers (running and stopped): **`docker ps --all`**

View resource usage stats: **`docker stats <container name/id>`**

This is just a sample set of commands with just one of the options. Students need to learn at least many more essential commands and all the associated flags or options for each of those essential commands for day to day work.

Container Image: A lightweight, standalone, executable, and **immutable** package of software that includes everything needed to run an application (code, runtime, libraries, environment variables, and config files). It is the static **blueprint** stored in a registry.

Container: A runtime instance of a container image. It is what a container image becomes when it is executed by a container engine (like Docker).

Build an Image from a Dockerfile: ***docker build -t <image name>***

Build an Image from a Dockerfile without the cache: ***docker build -t <image name>. --no-cache***

List local images: ***docker image ls / docker image ls --a (--all)***

Delete an Image: ***docker rmi <image name>***

Remove all unused images: ***docker image prune***

```
FROM python:3.11-slim
WORKDIR /app
RUN pip install flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Compile Image Blueprint:

```
docker build -t my-python-app .
```



Run the Compiled Custom Image:

```
docker run -d -p 5001:5000 --name running-python-container my-python-app
```

Verify: <http://localhost:5001> (from any browser)

(note, we will use port 5001 to connect from the external world. But, it maps to the internal port 5000)

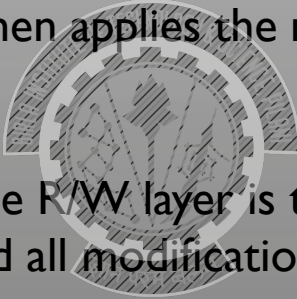
Dockerfile: File with instructions to build a docker container image

Layered Architecture: A container image consists of multiple read-only layers stacked on top of each other. Each instruction in a Dockerfile (like RUN or COPY) creates a new immutable layer.

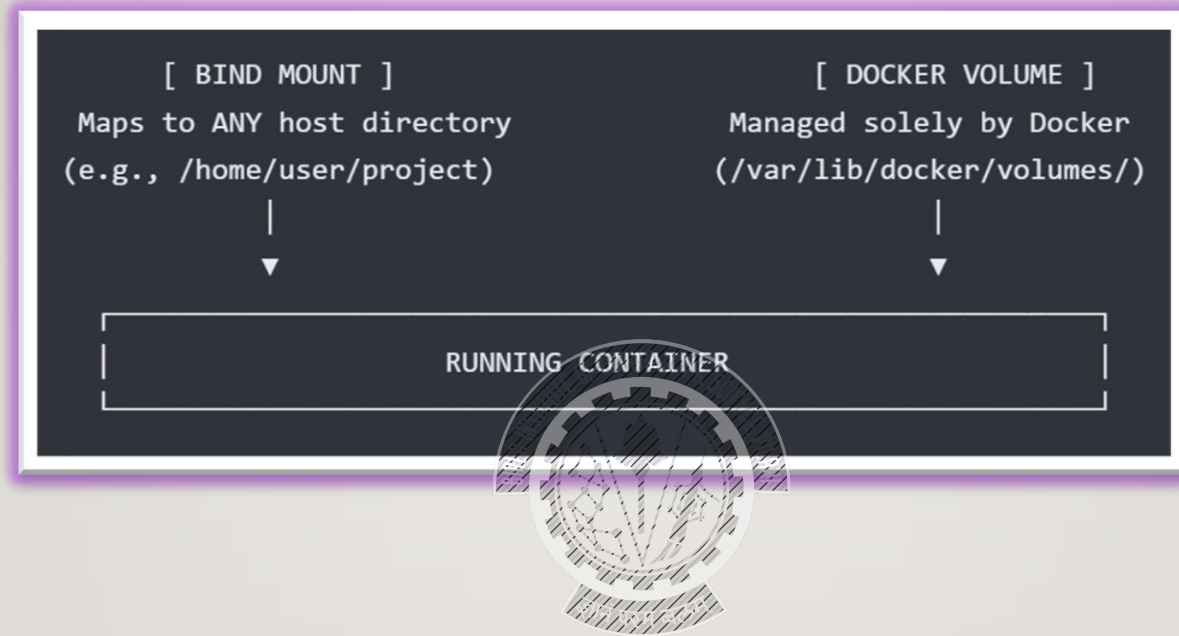
Ephemeral Writeable Layer: When a container is launched, Docker adds a thin, temporary **Read-Write (R/W) layer** on top of the image stack. Any changes made during runtime (creating, modifying, or deleting files) happen *only* in this top layer.

Copy-on-Write (CoW) Strategy: If a process inside a container modifies an existing file from the read-only image, Docker automatically copies the file up to the R/W layer first, then applies the modification. The original file in the lower image layers remains unchanged.

Storage Performance & Ephemerality: Because the R/W layer is tied directly to the container's lifecycle, **deleting the container permanently deletes the R/W layer** and all modifications inside it. Persistent data must use volumes or bind mounts.



Work Integrated Learning Programmes



Primary difference lies in the **management and portability**:

- Volumes are fully managed by Docker and isolated from host dependencies.
- Bind Mounts map directly to any arbitrary directory on the host system.

Docker uses network drivers to manage how containers communicate with each other, with the host machine, and with the outside world.

Three most essential networking modes.

- Bridge Network (The Default) - A Bridge network is a private, isolated virtual network created by Docker on the host machine.
- Host network - removes the network isolation between the container and the physical host machine.
- Overlay Network - An Overlay network connects multiple distinct physical Docker hosts together, creating a unified distributed network.

KEY CONCEPTS

Docker is an operating system for containers. Docker ensures that applications run the same way in development, testing, and production environments.

Docker **Images are Immutable**, pre-built packages that contain everything needed to run an application. Docker uses images to create containers

Containers are Isolated environments that run software without affecting the rest of the system. Containers are a standard unit of software that packages code and dependencies to run reliably across computing environments.

Docker containers can run on any system that supports Docker, whether it's a developer's laptop, a physical server, or a cloud environment.

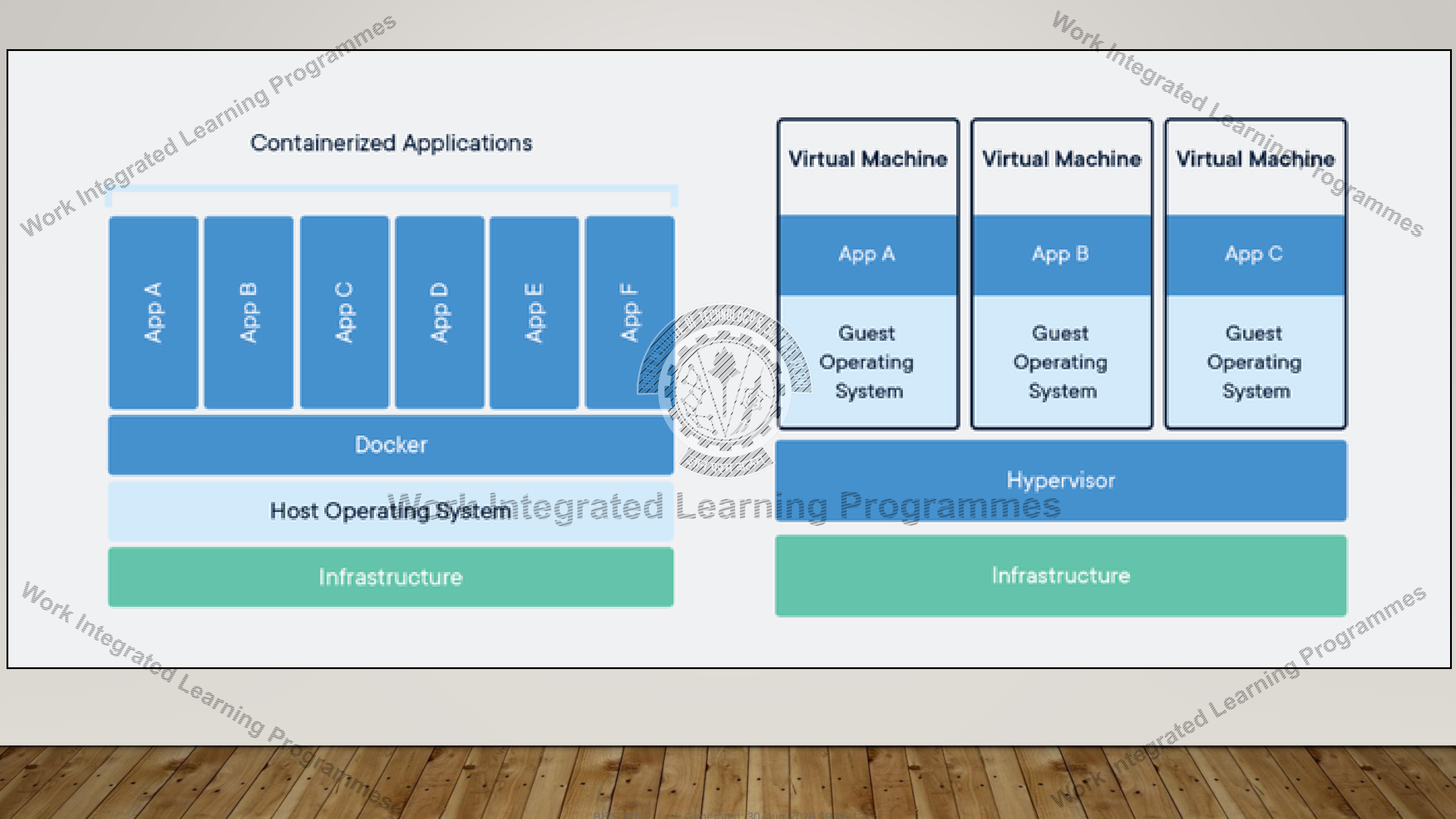
Containers are mutable, meaning they can be modified during runtime. Changes made within a container are isolated to that container

LET'S SEE A SCENARIO

A web application needs to be deployed across multiple environments (dev/test/staging/...)

Using Docker, we can realize this scenario

- Package the application: Create a Docker image that includes the application and its dependencies.
- Define the environment: Use Docker Compose to define the services, networks, and volumes required by the application.
- Deploy consistently: Deploy the same Docker image across development, testing, and production environments, ensuring consistency and reducing deployment errors.



There are so many container runtimes:

Docker
Podman
containerd
LXC/LXD
rkt (Rocket)

.....

With so many container runtimes, it definitely calls for standardization.



Work Integrated Learning Programmes

Standardization is needed

BIPS WILP | Generated: 30-Aug-2020 15:05:15

Open Container Initiative

The **Open Container Initiative** is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes.

Established in June 2015 by Docker and other leaders in the container industry, the OCI currently contains three specifications: the Runtime Specification (runtime-spec), the Image Specification (image-spec) and the Distribution Specification (distribution-spec). The Runtime Specification outlines how to run a "filesystem bundle" that is unpacked on disk. At a high-level an OCI implementation would download an OCI Image then unpack that image into an OCI Runtime filesystem bundle. At this point the OCI Runtime Bundle would be run by an OCI Runtime.

NAMESPACES – ORGANIZING AN ENTITY

Have you ever used Namespaces?

Can you tell an analogy in real life for the same?



Work Integrated Learning Programmes

NAMESPACES – C#

Namespaces in C# are a mechanism used to organize code into logical groups.

They help prevent naming conflicts, improve code readability, and facilitate code reuse.

Avoid Naming Conflicts: When you have multiple classes, methods, or variables with the same name, namespaces help differentiate them.

Organize Code: By grouping related classes and interfaces into namespaces, you can structure your code in a clear and logical manner.

Improve Code Reusability: Namespaces can be used to create libraries of reusable code that can be shared across multiple projects.

NAMESPACES – C#

```
namespace MyNamespace {  
// Code within this namespace  
}
```

```
MyNamespace.MyClass myObject = new MyNamespace.MyClass();
```

```
using MyNamespace;  
// Now we can use the class directly:  
MyClass myObject = new MyClass();
```

```
//example of nested spaces  
namespace OuterNamespace  
{  
    namespace InnerNamespace  
    {  
        // Code within the inner namespace  
    }  
}
```



Work Integrated Learning Programmes

WHY ARE WE SEEING C# EXAMPLES?

This is cloud computing class!!!

Just to make it easy for us to correlate when we go to the Namespace concepts for containers!



Work Integrated Learning Programmes

NAMESPACES – ORGANIZING AN ENTITY

Programming Namespaces are a context that provides a scope for identifiers, such as the names of variables, functions, and types. Namespaces help prevent name collisions, especially when a code base has multiple libraries. For example, in C#, class A is referenced with `global::A`, and the type `System.Net.Sockets.Socket` is referenced with `S::Socket`.

To understand container namespaces let us use a standard programming analogy - **Variable Shadowing**. This mimics how a container isolates an internal environment while the host system maintains its own external environment.

```
public class ScopeExample {  
    private string name = "Host-System";    //class scope  
    public void MyFunction()    {  
        string name = "Isolated-Container";    // 2. Method Scope (Like a Container Namespace)  
        Console.WriteLine(name);    // Prints "Isolated-Container" because the local variable shadows the class variable  
        Console.WriteLine(this.name);    // Prints "Host-System" because 'this' explicitly steps outside the local scope  
    }  
}
```



OTHER EXAMPLES

In operating systems

Directories are an example of namespaces in an operating system. Each name in a directory uniquely identifies a file or subdirectory. For example, the file foo.txt can exist in two different directories.... /home/cloudComputingSectionA and in /home/cloudComputingSectionB. But two copies of foo.txt cannot co-exist in the same directory.

In computer networks

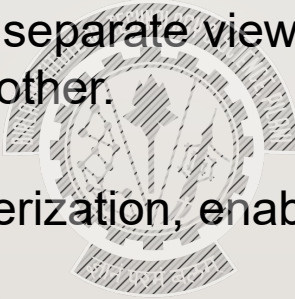
The internet is a single DNS name space, where all network devices with a DNS name can be resolved to a particular address.

NAMESPACES IN LINUX

Linux namespaces are a feature of the Linux kernel that provide isolation for system resources.

They allow different processes to have their own separate views of the system, ensuring that processes in one namespace cannot see or affect processes in another.

This is a fundamental technology behind containerization, enabling tools like Docker to create isolated environments.



Work Integrated Learning Programmes

TYPES OF NAMESPACES IN LINUX

PID Namespace: Isolates process IDs, so processes in different namespaces can have the same PID without conflict.

Network Namespace: Isolates network interfaces, IP addresses, routing tables, and other network-related resources.

Mount Namespace: Isolates the file system mount points, allowing different namespaces to have different views of the file system hierarchy.

UTS Namespace: Isolates hostname and domain name, allowing each namespace to have its own hostname.

IPC Namespace: Isolates inter-process communication resources like message queues, semaphores, and shared memory.

User Namespace: Isolates user and group IDs, allowing processes to have different user and group IDs in different namespaces.

Cgroup Namespace: Isolates control groups, which are used to limit and monitor resource usage like CPU, memory, and I/O.

LET'S USE AN ANALOGY AGAIN – APARTMENT BUILDING!

Let us take a large apartment building with multiple apartments. Each apartment represents a different namespace, and the building itself represents the entire system.

Building (System): The entire apartment building is like the whole computer system, hosting multiple apartments (namespaces).

Apartments (Namespaces): Each apartment is a separate namespace. Just like each apartment has its own rooms and facilities, each namespace has its own set of resources and processes.

Rooms (Resources): Inside each apartment, there are different rooms like the kitchen, living room, and bedroom. These rooms represent different types of resources such as process IDs (PID), network interfaces, and file systems.

Tenants (Processes): The tenants living in each apartment are like the processes running within a namespace. Tenants in one apartment cannot see or interfere with tenants in another apartment, just like processes in one namespace are isolated from those in another. Of course, if a tenant A invites another tenant or permits another tenant to view or use, it is a different story.

Building (System): The entire apartment building is the whole computer system, hosting multiple apartments (namespaces). The Building Manager (Host OS Kernel) oversees everything.

Apartments (Namespaces): Each apartment is an isolated environment with its own set of resources.

Tenants (Processes): The people living inside. Tenants in Apartment A cannot see or interfere with what Tenants in Apartment B are doing.

PID Namespace (Local Identity): Inside the apartment, tenants refer to the head of the household simply as "Mom" (PID 1). To the Building Manager outside, that same person is tracked by a unique global lease number or id.

Network Namespace (Wi-Fi): Each apartment has its own Wi-Fi router. Tenants manage their own local network, IP addresses, and routing without conflicting with the neighbour's network.

Mount Namespace (Floor Plan/Cabinets): Each apartment has its own unique layout of closets and cabinets (file system). A tenant can lock, rearrange, or hide a cabinet without affecting the layout of any other apartment.

UTS (UNIX Time Sharing) Namespace (Nameplate): Each apartment can have its own distinct nameplate on the front door (hostname and domain name).

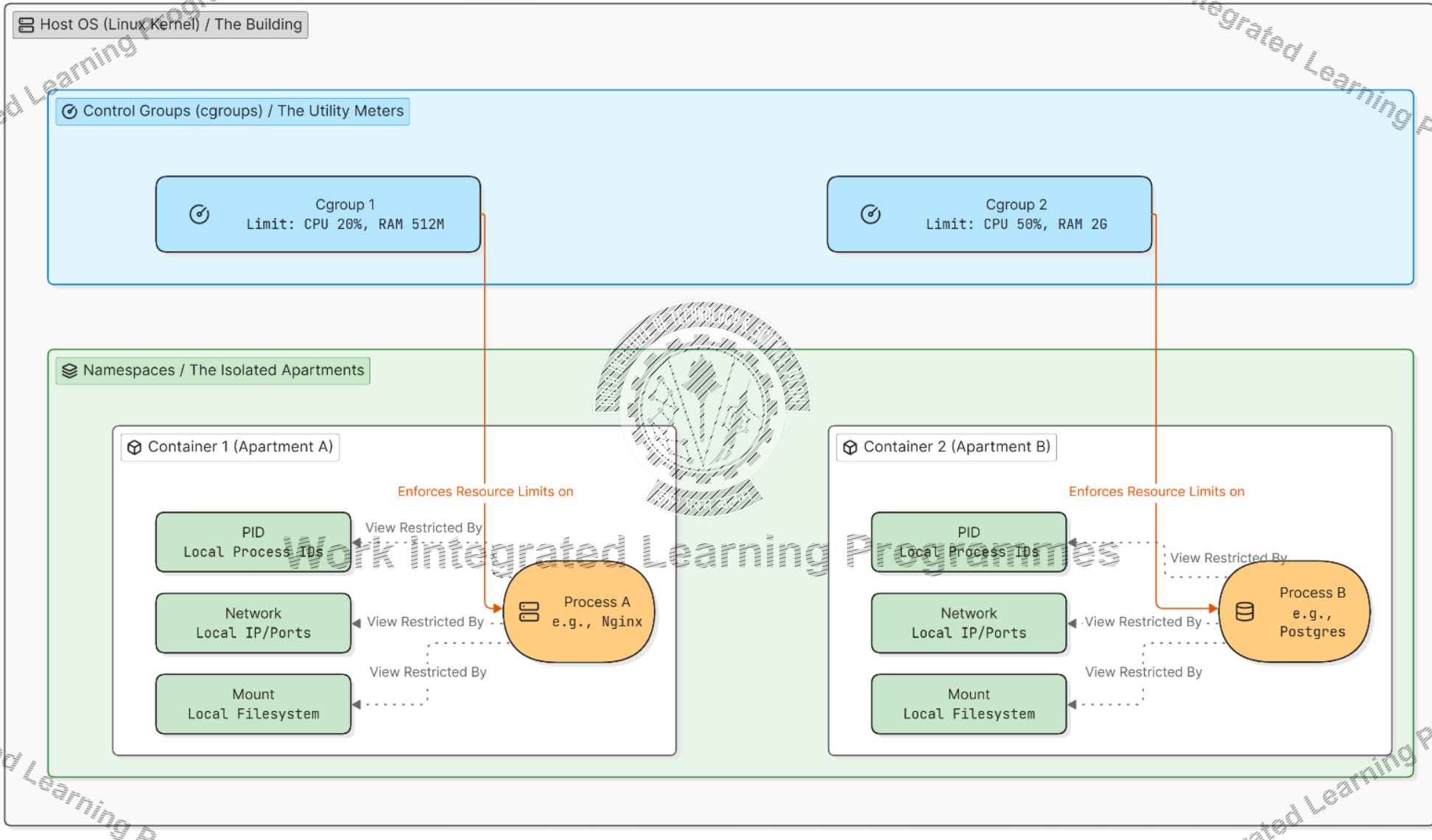
IPC Namespace (Intercom): Each apartment has its own internal intercom system. Tenants can chat via the intercom (message queues, semaphores) without their conversations being broadcast to other apartments.

User Namespace (House Rules): A tenant might be the "Master of the House" (root user) inside their own apartment, able to do whatever they want. But in the building's main lobby, the Manager treats them as a regular, restricted tenant (unprivileged user).

Cgroup Namespace (Meter Box): Ensures the tenant can only see the utility meter for their own apartment, hiding the usage data of the rest of the building.

Note: Cgroups (Utility Meters): The building sets a hard cap on how much water and electricity (CPU and RAM) each apartment can consume to prevent one tenant from draining the building's supply.

Linux Containers: Namespaces and Cgroups



The Horizontal Walls (Namespaces): Focus on how Process A and Process B are completely boxed into their own environments. The namespaces dictate **what the process can see**. Process A looks around and only sees its local PIDs, network interfaces, and file mounts.

The Vertical Limiters (Cgroups): The Cgroups sit above the processes and act as hard pipelines. They dictate **what the process can use**. Even if Process A tries to consume 100% of the host's CPU, the Cgroup acts as a valve, throttling it down to the allowed 20% without the process necessarily understanding why it is running slower!

The Intersection: A container is essentially just a standard Linux process (the orange circles) that has been wrapped in both a Namespace (for invisibility to neighbors) and a Cgroup (for resource starvation prevention).

LXC

LXC (Linux Containers): LXC is a low-level user space interface for the Linux kernel's containment features. It provides tools, templates, and libraries to create and manage containers.

- Isolation: LXC containers offer process and network isolation, making them lightweight alternatives to virtual machines.
- Usage: Ideal for users who need fine-grained control over container configurations and are comfortable with command-line tools

LXD

Daemon on Top of LXC: LXD is a system daemon that builds on top of LXC, providing a more user-friendly and intuitive way to manage containers.

- LXD offers additional features like REST API, image management, and network management, making it easier to operate containers at scale.
- Suitable for users who want a more streamlined and manageable approach to container orchestration, with capabilities similar to virtual machine management.
- LXD has a graphical user interface (UI). Starting with LXD 5.21.0 LTS, an official LXD UI tool is available by default. Interface: Browser based

In summary, LXC is the underlying technology for creating containers, while LXD provides a higher-level interface to manage these containers more efficiently and with additional features.

Reference for comparison with docker <https://ubuntu.com/blog/lxd-vs-docker>

LXC VS DOCKER

With LXC, we can configure detailed network settings for each container. For instance, we can specify the type of network interface, the link it connects to, and even assign specific IP addresses and routes. Note: The example below provides an idea of fine grained control. If it is not clear for you, it is ok to skip. Just understand that this gives even more fine grained control.

```
lxc.net.0.type = veth; lxc.net.0.link = br0; lxc.net.0.flags = up; lxc.net.0.hwaddr = 00:16:3e:xx:xx:xx;
```

```
lxc.net.0.ipv4.address = 192.168.1.100/24; lxc.net.0.ipv4.gateway = 192.168.1.1; lxc.net.0.ipv6.address = 2001:db8::100/64
```

```
lxc.net.0.ipv6.gateway = 2001:db8::1
```

`lxc.net.0.type` specifies the type of network interface (veth). `lxc.net.0.link` connects the container to a bridge (br0). `lxc.net.0.ipv4.address` and `lxc.net.0.ipv6.address` assign specific IP addresses to the container. `lxc.net.0.ipv4.gateway` and `lxc.net.0.ipv6.gateway` set the gateways for IPv4 and IPv6.

No need to understand this thoroughly. Just get an idea that LXC offers fine grained control over docker containers.

LXC VS DOCKER

LXC – High performance and resource efficiency, running legacy apps, server consolidation,...

Docker - Rapid application development and deployment, Microservices architecture, portable and consistent, developer friendly ecosystem....



Work Integrated Learning Programmes